

This is a copy of a shared ChatGPT conversation
Report conversation

In Django, test cases are usually written using Django's built-in `TestCase` class. Tests help verify that your models, views, APIs, and business logic work correctly.

Step 1: Create a `tests.py` file

Each Django app normally has a `tests.py` file.

Example:

```
api/  
├─ models.py  
├─ views.py  
└─ tests.py
```



Step 2: Import `TestCase`

</> Python



Run

Step 3: Write a simple model test

Suppose you have this model:

```
class Product(models.Model):
    name = models.CharField(max_length=100)
    weight = models.FloatField()
```

Test:

```
from django.test import TestCase
from .models import Product

class ProductTest(TestCase):

    def test_create_product(self):
        product = Product.objects.create(
            name="Laptop",
            weight=2.5
        )

        self.assertEqual(product.name, "Laptop")
        self.assertEqual(product.weight, 2.5)
```

Step 4: Test an API endpoint

If your API returns product details:

```
from django.test import TestCase
from django.urls import reverse
from .models import Product
```



```
class ProductAPITest(TestCase):

    def setUp(self):
        Product.objects.create(
            name="Laptop",
            weight=2.5
        )

    def test_product_api(self):
        response = self.client.get(reverse("product-list"))

        self.assertEqual(response.status_code, 200)
```

Step 5: Run the tests

From the project folder (where `manage.py` is located):

 **Bash**



If all tests pass, you'll see output similar to:

Ran 2 tests in 0.15s



OK

Test cases for your AI-Assisted Box Selection project

For your project, these are good tests to include:

1. Product model

```
def test_product_creation(self):
    product = Product.objects.create(
        name="Laptop",
        length=30,
        width=20,
        height=5,
        weight=2
```

```
)  
  
self.assertEqual(product.name, "Laptop")
```

2. Box model

```
def test_box_creation(self):  
    box = Box.objects.create(  
        name="Medium Box",  
        max_weight=5  
    )  
  
    self.assertEqual(box.name, "Medium Box")
```

3. API response

```
response = self.client.get("/api/products/")  
self.assertEqual(response.status_code, 200)
```

4. Box recommendation logic

Create products and boxes, call the recommendation endpoint, and verify:

- A box is recommended.
- The selected box is the expected one.
- The response returns HTTP 200.

Example:

```
response = self.client.get("/api/recommend/1/")  
self.assertEqual(response.status_code, 200)  
self.assertIn("recommended_box", response.json())
```

What interviewers expect

If asked "**Why do we write test cases?**", you can answer:

"Test cases help verify that the application works correctly and continue to work after future changes. They allow us to catch bugs early, validate business logic, and ensure APIs and models behave as expected."

For a Django hiring assignment, even **3–5 well-written tests** (covering models, APIs, and the core recommendation logic) demonstrate good development practices.

Is my project folder structure correct?

I can check it, but I'll need to see your project structure.

Please do **one** of these:

1. **Upload a screenshot** of your project folder (VS Code Explorer is ideal), **or**
2. Run this command in your project folder and paste the output:

Windows (PowerShell):

</> PowerShell



or

</> PowerShell



For a typical Django project like your **AI-Assisted Box Selection System**, the structure should look something like this:

```
box_selector/  
|— manage.py  
|— db.sqlite3
```



```

|— requirements.txt
|— README.md
|— .gitignore
|
|— box_selector/
|   |— __init__.py
|   |— settings.py
|   |— urls.py
|   |— asgi.py
|   |— wsgi.py
|
|— api/
|   |— migrations/
|   |   |— __init__.py
|   |— __init__.py
|   |— admin.py
|   |— apps.py
|   |— models.py
|   |— urls.py
|   |— views.py
|   |— tests.py
|   |— serializers.py    (if using Django REST Framework)
|
|— venv/                (optional, usually not pushed to GitHub)

```

Before submitting, make sure you have:

- ☒ manage.py
- ☒ Project folder (settings.py , urls.py , etc.)
- ☒ App folder (models.py , views.py , admin.py)
- ☒ requirements.txt
- ☒ README.md
- ☒ .gitignore
- ☒ tests.py (recommended)
- ☒ migrations/ folder with __init__.py

Upload a screenshot of your folder structure, and I'll tell you if anything is missing or should be reorganized.

How can I improve my project documentation?

Good documentation makes your project easier to understand and leaves a positive impression on recruiters and reviewers. For your **AI-Assisted Box Selection System**, your README.md should cover the following sections.

1. Project Title

</> Markdown



2. Project Overview

Briefly explain what the project does.

```
## Overview
```

```
This Django application recommends the most suitable shipping box for customer orders based on
```

3. Features

List the main capabilities.

```
## Features
```

- Product management
- Shipping box management
- Order creation
- Automatic box recommendation
- REST API support
- Django Admin interface

4. Tech Stack

Mention the technologies used.

Tech Stack

- Python
- Django
- Django REST Framework
- SQLite
- Git & GitHub

5. Installation

Provide clear setup instructions.

Installation

```
```bash
git clone <repository-url>
cd box_selector
python -m venv venv
```
```

Activate the virtual environment.

Install dependencies:

```
```bash
pip install -r requirements.txt
```
```

Run migrations:

```
```bash
python manage.py migrate
```
```

Start the server:

```
```bash
python manage.py runserver
```
```

6. API Endpoints

Document the available endpoints.

```
## API Endpoints

GET /api/products/

GET /api/boxes/

GET /api/recommend/<product_id>/

GET /api/order-recommend/<order_id>/
```

If applicable, include example requests and responses.

7. Project Structure

Help readers understand the code layout.

```
box_selector/
|— manage.py
|— api/
|— box_selector/
|— requirements.txt
|— README.md
```



8. Testing

Explain how to run the tests.

```
## Running Tests

```bash
python manage.py test
```
```

9. AI Usage

If your assignment requests AI usage documentation, include a short section explaining how AI assisted you and confirm that you reviewed and tested the final implementation.

10. Future Improvements

Show that you've thought beyond the current scope.

Examples:

- User authentication
 - Better packing algorithms
 - PostgreSQL support
 - Docker deployment
 - Admin dashboard with analytics
-

11. Author

</> Markdown



Author

Pournima Mali

Tips to make it even better

- Add **screenshots** of the Django Admin panel and any API responses.
- Include a **sample dataset** or explain how to create test data.
- Explain any **assumptions** your recommendation algorithm makes (for example, how it chooses between multiple suitable boxes).
- Keep the language clear and concise.
- Use headings, bullet points, and code blocks for readability.

A README with these sections is professional, easy to follow, and gives reviewers everything they need to understand, run, and evaluate your project.

ChatGPT can make mistakes. Check important info.